



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Yazılım Geliştirme I

Fonksiyonlar ve Bellek Modeli

Hafta 2

Fonksiyonlar ve Bellek Modeli

Doç. Dr. Tolga Berber

Bilgisayar Bilimleri · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları:

1 • Ön-öğretim

2 • İçerik

3 • Katılım

4 • Değerlendirme

5 • İzleme

1 • Ön-öğretim Neden Önemli?

Bir fonksiyona bir **Nokta** verdiniz; fonksiyon onu “öteledi”, ama **main**’e döndüğünüzde **Nokta** hiç değişmemişti — ya da tam tersi, hiç beklemediğiniz hâlde değişmişti. Uzun çalışan bir programın belleğinin durmadan şişip sonunda tükenmesi de aynı ailedendir.

- Bu davranışların tek nedeni, parametreyi hangi biçimde geçtiğiniz ve belleğin “otomatik mi, sizin mi” olduğudur
- Bu hafta “bir değer kopyalanır mı, değişir mi; belleği geri vermeyi kim üstlenir?” sorularını bilinçle yanıtlayacaksınız

Bu Hafta

© Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

- 1 Aşırı yükleme ve varsayılan parametre tanımlayabileceksiniz
- 2 Değer/referans/const referans/işaretçi ile parametre geçme arasındaki farkı seçebileceksiniz
- 3 Yığın vs öbek, new/delete, nullptr kullanabileceksiniz
- 4 Bellek sızıntısı ve sarkan işaretçiyi tanıyabileceksiniz
- 5 RAIİ fikrini ilk bakış düzeyinde uygulayabileceksiniz

BÖLÜM 1

Ünite 1 — Aşırı Yükleme ve Varsayılan Parametre

Aşırı Yükleme Parametre Geçme Bellek Modeli RAIİ Özet

2 • İçerik Aynı Ad, Farklı İmza

C'de bir fonksiyon adı programda **benzersiz** olmak zorundadır — kare/dikdörtgen/daire alanı için `alanKare`, `alanDikdortgen`, `alanDaire` gibi üç ayrı ad uydurmanız gerekir.

Tanım: Aşırı yükleme

Aynı adı taşıyan, parametre listesi (sayı/tip) farklı birden çok fonksiyon tanımlama olanağı. Derleyici çağrı anındaki argümanlara bakarak hangi sürümün kastedildiğine karar verir.

Analoji: “Aç” Fiili

Analoji

“Kapıyı aç”, “dosyayı aç”, “mağazayı aç” — aynı fiili kullanırsınız, ama yanındaki isme (parametreye) bakarak dinleyici hangi eylemi kastettiğinizi otomatik çıkarır. Derleyici de `alan(...)` çağrısını gördüğünde argümanların sayı/tipine bakarak hangi `alan` sürümünün kastedildiğini aynı şekilde çözer.

Aşırı Yükleme: Kod

```
5 int alan(int kenar) {  
6     return kenar * kenar;  
7 }  
8  
9 int alan(int taban, int yukseklik) {  
10     return taban * yukseklik;  
11 }  
12  
13 double alan(double yaricap) {  
14     return 3.14159 * yaricap * yaricap;  
15 }
```

Üçü de `alan` adını taşır; `alan(4)`, `alan(3, 5)`, `alan(2.0)` çağrılarının her biri yalnızca bir imzayla eşleşir — karar tamamen derleme anında verilir.

İmza: Dönüş Tipi Sayılmaz

Tanım: İmza

Bir fonksiyonun *imzası* (*signature*), ad + parametre listesi bütünüdür. **Dönüş tipi imzanın parçası değildir.**

Dikkat

Yalnızca dönüş tipiyle ayrılan iki fonksiyon **derlenmez**: `int alan(int kenar);` ile `double alan(int kenar);` aynı imzaya sahiptir — derleyici `alan(4)` çağrısında hangisinin çalışacağını belirleyemez.

Varsayılan Parametreler

Analoji

“Bir kahve” dediğinizde boy belirtmezseniz **varsayılan** (orta) boy gelir; isterseniz geçersiz kılarırsınız.

```
19 long guc(int taban, int us = 2) {  
20     long sonuc = 1;  
21     for (int i = 0; i < us; ++i) {  
22         sonuc *= taban;  
23     }  
24     return sonuc;  
25 }
```

$\text{guc}(3) \rightarrow 3^2 = 9$ (varsayılan); $\text{guc}(3,3) \rightarrow 3^3 = 27$. Varsayılanlı parametreler listenin **sonunda** olmalıdır.

BÖLÜM 2

Ünite 2 — Değer, Referans, const Referans, İşaretçi

Aşırı Yükleme Parametre Geçme Bellek Modeli RAIİ Özet

2. İçerik Değere Göre Geçme: Kopya Alırsınız

Analoji

Değere göre geçmeyi bir belgenin fotokopisini vermeye benzetebilirsiniz — fotokopinin üzerine ne yazılırsa yazılsın elinizdeki orijinal değişmez.

```
8 void oteleDeger(Nokta n) {  
9     n.x += 10.0;  
10    n.y += 10.0;  
11 }
```

`n`, çağırandaki nesnenin **ayrı bir kopyasıdır** — gövdedeki değişiklik çağırana hiç yansımaz.

Referansla Geçme: Takma Ad Verirsiniz

Analoji

Referansla geçmeyi, fotokopi yerine belgenin orijinalini elden vermeye benze-
tebilirsiniz — karşı taraf belgenin kendisiyle çalışır, her değişiklik kalıcıdır.

```
15 void oteleReferans(Nokta& n) {  
16     n.x += 10.0;  
17     n.y += 10.0;  
18 }
```

Kopya **yoktur**; **n** çağırandaki nesneye doğrudan bağlıdır — gövdedeki değişiklik çağırana yansır.

const Referans: Kopyasız + Değiştirilemez

```
22 double oteleConstReferans(const Nokta& n) {  
23     // yalnızca okur; n üzerinde değişiklik yapamaz (const)  
24     return n.x + n.y;  
25 }
```

- & kopyayı önler, `const` yazmayı yasaklar
- Büyük bir nesneyi **yalnızca okumak** için bu haftanın varsayılan tercihi

İşaretçiyle Geçme: Adres Verirsiniz

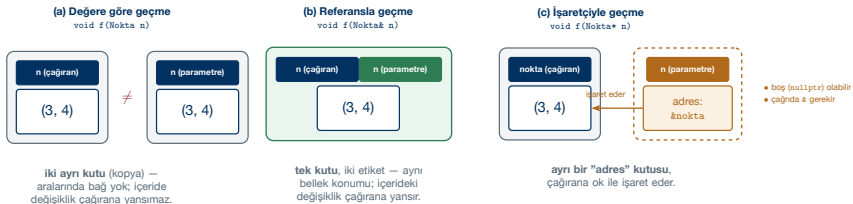
Analoji

Belgenin saklandığı dolabın *numarasını* vermeye benzer — boş olabilir, kontrol şarttır.

```
29 void oteleIsaretci(Nokta* n) {  
30     if (n == nullptr) {  
31         return;  
32     }  
33     n->x += 10.0;  
34     n->y += 10.0;  
35 }
```

Kullanmadan önce **her zaman** `nullptr` kontrolü yapın.

Üç Biçim Bellek Düzeyinde



Değer: iki ayrı kutu. Referans: tek kutu, iki isim. İşaretçi: ayrı adres kutusu, boş olabilir.

Karar Tablosu

Durum	Tercih
Küçük tip (int, Nokta)	Değere göre (T)
Büyük tip, değişecek	Referansla (T&)
Büyük tip, yalnız okunacak	const referansla (const T&)
Parametre isteğe bağlı/boş olabilir	İşaretçiyle (T*)

BÖLÜM 3

Ünite 3 — Yığın vs Öbek, new/delete

Aşırı Yükleme Parametre Geçme **Bellek Modeli** RAII Özet

2. İçerik **Yığın (Stack): Otomatik Ömür**

Tanım: Yığın (stack)

Her fonksiyon çağrısında büyüyüp geri döndüğünde küçülen bellek bölgesi. Yerel değişkenler otomatik ayrılır, kapsam bitince otomatik geri alınır.

```
8 void stackOrnegi() {  
9     Nokta yerel{5.0, 5.0};  
10    std::cout << "stack nesnesi adresi: " << &yerel << '\n';  
11 }
```

`stackOrnegi()` döndüğünde `yerel`'in belleği otomatik geri alınır — o adres artık geçersizdir.

Öbek (Heap): new/delete

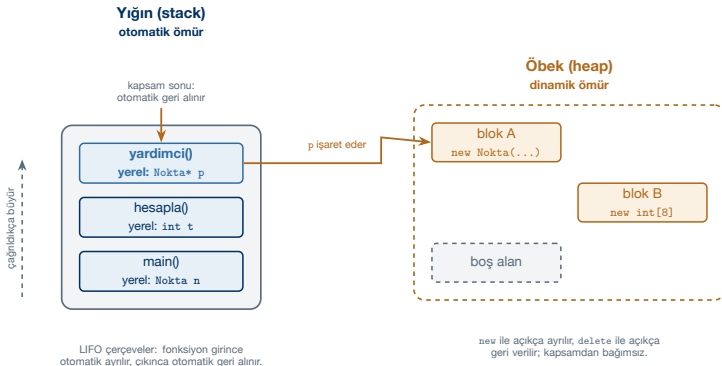
Tanım: Öbek (heap)

`new` ile açıkça istenip `delete` ile geri verilen, kapsamdan bağımsız bellek bölgesi.

```
15 void newDeleteOrnegi() {  
16     Nokta* dinamik = new Nokta{7.0, 7.0};  
17     std::cout << "dinamik nesne adresi: " << dinamik << '\n';  
18     delete dinamik;  
19     dinamik = nullptr;  
20 }
```

C'nin `malloc/free`'sine kıyasla boyutu kendisi hesaplar, sonuç doğru tiptedir.

Yığın vs Öbek



Yığın masaüstünüz gibidir — iş bitince kendiliğinden toplanır. Öbek kiraladığınız depodur — kiralamadıkça (**new**) size ait olmaz, iade etmedikçe (**delete**) boşalmaz.

nullptr: Tip Güvenli Boş İşaretçi

C'nin `NULL/0` makrosuna kıyasla `nullptr` yalnızca işaretçi bağlamında geçerlidir.

```
24 void nullptrKontrolOrnegi() {  
25     Nokta* p = nullptr;  
26     if (p == nullptr) {  
27         std::cout << "isaretci bos (nullptr), erisim yapilmiyor\n";  
28     } else {  
29         std::cout << "isaretci dolu: " << p->x << '\n';  
30     }  
31 }
```

İpucu: bir işaretçiyi hemen değer veremiyorsanız bile `nullptr` ile ilkleyin.

Bellek Sızıntısı: Unutulan delete

```
35 // DİKKAT: bu fonksiyon KASITLI olarak hatalıdır; main() tarafından  
    CAGRILMAZ.  
36 // new ile ayrılan belleğin karşılığı hiçbir zaman delete edilmiyor.  
37 void sızıntıOrnegi() {  
38     Nokta* p = new Nokta{1.0, 1.0};  
39     p = new Nokta{2.0, 2.0}; // ilk ayrılan adres burada kaybolur,  
    artık delete edilemez  
40     // delete p; // <- kasitli olarak cagrilmadi: bellek sızıntısı bu  
    satirin eksikliginden dogar  
41     (void)p;  
42 }
```

İkinci `new` sonucu `p`'ye atanınca birinci nesnenin adresi kaybolur — o bellek program sonlanana kadar geri alınamaz.

Sarkan İşaretçi

```
46 // DIKKAT: bu fonksiyon KASITLI olarak sarkan isaretci DESENINI gosterir;  
47 // main() tarafından CAGRILMAZ ve p üzerinden gercek bir erisim (dereference)  
   YAPILMAZ.  
48 void sarkanOrnegi() {  
49     Nokta* p = new Nokta{3.0, 3.0};  
50     delete p;  
51     // p artik sarkan (dangling) bir isaretci: delete edilmiş belleği gösteriyor.  
52     // İyi aliskanlik: delete sonrasi "p = nullptr;" yapmaktır (burada bilerek  
       atlandı).  
53     // p->x = 9.0; // <- YAPILMAMALIDIR: tanımsız davranış (UB); bu satir hicbir  
       zaman calismaz.  
54 }
```

`delete p;` sonrası `p` hâlâ eski adresi taşır — bu andan itibaren **sarkan işaretçidir**, kullanılması tanımsız davranıştır.

BÖLÜM 4

Ünite 4 — RAII'ye İlk Bakış

Aşırı Yükleme Parametre Geçme Bellek Modeli **RAII** Özet

2. İçerik Elle Yönetimin Gerçek Riski

```
7   std::cout << "-- elle yonetim --\n";
8   int* elleTampon = new int[5];
9   for (int i = 0; i < 5; ++i) {
10      elleTampon[i] = i * i;
11  }
12  std::cout << "elle tampon dolduruldu\n";
13  // Not: burada delete[] cagrisi UNUTULURSA bellek sızıntısı oluşur
14
15  delete[] elleTampon;
16  std::cout << "elle tampon serbest bırakıldı\n";
```

Bu kod doğru görünür — ama doğruluk tamamen **sizin disiplininize** bağlıdır. Doldurma döngüsünün ortasına bir erken **return** eklense, **delete[]** satırına hiç uğranmaz: dizi sessizce sızar.

RAII Fikri

Tanım: RAI

Bir kaynağın edinim/bırakım işlemlerini bir nesnenin ömrüne bağlayan ilke: kaynak nesne **oluşturulduğunda** edinilir, bırakılması nesnenin **kapsam sonunda** **otomatik çalışan** özel bir fonksiyona bağlanır.

Analoji

Bir otel oda kartı gibidir: kartı check-in'de alırsınız, oda sizindir. Kartı iade etmeyi unutsanız bile otel, süreniz dolunca kapıyı kendiliğinden kilitler.

araclar::IntTampon: Yapı

```
8 | int* veri;  
9 | int boyut;
```

```
12 | IntTampon(int n);  
13 | ~IntTampon();
```

`IntTampon(int n)` nesne oluşturulduğunda, `~IntTampon()` nesne kapsam dışına çıktığında derleyici tarafından otomatik çağrılır — bu haftada yalnız bu davranış tanınır, biçimsel terimler Hafta 3'te.

IntTampon: Gövdeler

```
8 IntTampon::IntTampon(int n) {  
9     veri = new int[n];  
10    boyut = n;  
11    std::cout << "tampon olusturuldu (boyut=" << boyut << ")\n";  
12 }
```

```
16 IntTampon::~~IntTampon() {  
17     delete[] veri;  
18     std::cout << "tampon temizlendi\n";  
19 }
```

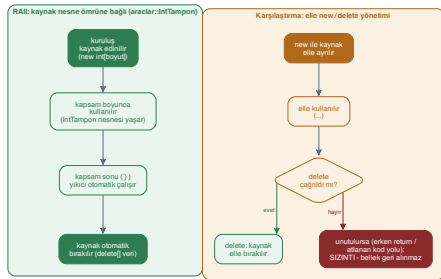
İkinci gövde elle yönetimde siz hiç yazmadığınız hâlde **otomatik** çalışan kapanış adımıdır — bir daha unutulamaz.

Elle Yönetim vs RAIİ: Yan Yana

```
19     std::cout << "-- RAIİ demo --\n";  
20     {  
21         araclar::IntTampon tampon(5);  
22         for (int i = 0; i < tampon.boyut; ++i) {  
23             tampon.veri[i] = i * i;  
24         }  
25         std::cout << "tampon kullaniliyor\n";  
26     } // blok sonu: yikici otomatik calisir, tampon kendini temizler  
27     std::cout << "blok disina cikildi\n";
```

Blok sonunda — siz hiçbir `delete[]` yazmadan — yıkıcı otomatik çağrılır; “tampon temizlendi” konsola düşer.

RAIİ: Kaynak Ömrü



Kuruluş: kaynak edinilir. Kapsam sonu: kaynak otomatik bırakılır; **delete** unutulursa yol sızıntıyla biter.

Not: `IntTampon` içinde hâlâ elle yazılmış bir `new/delete` çifti vardır — RAIİ bunu kaldırmaz, yalnızca tek bir yere taşır.

BÖLÜM 5

Özet

Aşırı Yükleme Parametre Geçme Bellek Modeli RAI **Özet**

Özet

Özet

- Aşırı yükleme: aynı ad, farklı imza (parametre listesi) — dönüş tipi imzaya dahil değil; varsayılan parametre atlanabilir argüman sağlar
- Değere geçme kopyalar; referans/işaretçi kopyasız değiştirir; `const T&` kopyasız + değiştirilemez okur
- Yığın otomatik/kapsam bağımlı; öbek `new/delete` ile elle yönetilir — unutma sızıntıya, yanlış kullanım sarkan işaretçiye yol açar
- RAll: kaynağın bırakılmasını nesnenin kapsam sonuna bağlar — elle hatırlamaya gerek kalmaz

5 • İzleme Aktarım — Başka Nerede?

Bu haftanın kararları **Nokta/IntTampon** örneklerine özgü değildir; her değer geçtiğinizde ve her dinamik bellek kullandığınızda aynen işler:

- Karar tablosu (küçük tip $\rightarrow$ değere; büyük, değişecek $\rightarrow$ **T&**; büyük, salt okunacak $\rightarrow$ **const T&**; opsiyonel $\rightarrow$ **T***) her fonksiyon imzasında yeniden karşınıza çıkar
- İlerideki Kitaplık projesinde bir **Kitap**'ı listeye eklerken gereksiz kopyalamamak için **const Kitap&**, yerinde güncellemek için **Kitap&** ile geçmek tam da bu kararın uygulamasıdır
- “Kaynağı nesnenin ömrüne bağlama” fikri yalnız belleğe özgü değildir: açık bir dosya, bir ağ bağlantısı ya da bir kilit de aynı desenle yönetilir — Hafta 3–4'te bu fikir olgunlaşacak

Sırada Ne Var?

Bu hafta lab'da

`geometri::Nokta` üzerinde dört parametre geçme biçimini *kendi elinizle* yazacak, `araclar::IntTampon` ile RAI'yi elle sarmalayacaksınız.

- Ders notundaki **Kendinizi Sınavın** soruları her üniteyi kapsıyor: imza neden dönüş tipini içermez, değer/referans farkı, unutulmuş `delete`'in sonucu, RAI'nin azalttığı risk
- Gelecek hafta: veriyi (alanlar) ve davranışı (üye fonksiyonlar) bir sınıfın çatısı altında birleştirmeyi — **kapsülleme**, kurucu/yıkıcı, `this` — göreceğiz